

BARATI GHARATI — Smart AI-Driven Full-Stack Wedding Planning Platform

Complete Technical Architecture Report, Working Flow & Presentation Material

■ Executive Summary

Barati Gharati is an end-to-end, enterprise-grade digital ecosystem designed to streamline wedding event planning, vendor matching, multi-date availability scheduling, lead management, and payment processing.

Built using modern web technologies (**Next.js**, **NestJS**, **Prisma ORM**, **PostgreSQL**, and **Razorpay**), the platform connects couples (**Customers**), event service providers (**Vendors**), and system supervisors (**Admins**) through an intelligent automated booking workflow.

■ 1. Complete Working Flow of the Smart Vendor Booking System

```
sequenceDiagram
    autonumber
    actor Customer
    participant Frontend as Frontend (Next.js)
    participant Engine as Booking Engine (NestJS)
    participant DB as Database (PostgreSQL)
    actor PrimaryVendor as Primary Vendor
    actor StandbyVendors as Standby Vendors
    participant Razorpay as Razorpay Gateway

    Customer->>Frontend: Select Package & Pick Multiple Event Dates
    Frontend->>Engine: POST /api/v1/bookings (Payload + eventDates[])
    Engine->>DB: 1. Create Booking Record (Status: PENDING)
    Engine->>Engine: 2. Check 100% Date Availability for Requested Vendor

    alt Requested Vendor is Free on ALL Selected Dates
        Engine->>DB: Create PRIMARY Assignment (Status: PENDING, SLA: 2 Hours)
        Engine->>DB: Update Booking Status -> WAITING_PRIMARY_VENDOR
        Engine->>PrimaryVendor: Send Email: "Urgent: Primary Request #WD2026..."
    end

    Engine->>Engine: 3. Discover & Score Standby Candidates in Category
    Engine->>DB: Create STANDBY Assignments for Eligible Vendors
    Engine->>StandbyVendors: Send Broadcast Email: "New Standby Opportunity"

    alt Primary Vendor Accepts within 2 Hours
        PrimaryVendor->>Engine: Response: ACCEPT
        Engine->>DB: Update Status -> WAITING_PAYMENT
        Engine->>Customer: Email: "Vendor Accepted! Pay Advance to Lock Date"
        Customer->>Razorpay: Pay Booking Advance Amount
        Razorpay-->>Engine: Signature Verification (HMAC-SHA256)
        Engine->>DB: Update Status -> ADVANCE_PAID / CONFIRMED
        Engine->>DB: Lock Requested Event Dates on Vendor Calendar
    else Primary Vendor Rejects (Mandatory Reason) OR Times Out (2 Hours)
        PrimaryVendor->>Engine: Response: REJECT (Reason: "■ Already Booked")
        Engine->>DB: Mark Primary -> REJECTED / TIMED_OUT
        Engine->>Engine: Promote Top Standby Vendor to PRIMARY
        Engine->>DB: Update Promoted Vendor -> PRIMARY (SLA: 2 Hours)
        Engine->>StandbyVendors: Send Email: "■ You've Been Promoted to Primary!"
    end

    opt Customer Initiates Vendor Swap
        Customer->>Engine: POST /api/v1/bookings/swap (Select Standby Vendor)
        Engine->>DB: Demote Old Primary -> STANDBY
        Engine->>DB: Promote Selected Vendor -> PRIMARY
        Engine->>PrimaryVendor: Email: "Role Updated: Standby"
        Engine->>StandbyVendors: Email: "Role Updated: Primary Vendor"
    end
```

Detailed Step-by-Step Breakdown

Step 1: Multi-Date Event Selection (Frontend)

1. Customer browses vendor profile and selects package.

2. Customer selects 1 or more event dates (e.g., *Haldi*: 24 Aug, *Sangeet*: 25 Aug, *Wedding*: 26 Aug).
3. Customer submits form. Frontend invokes **POST /api/v1/bookings** with **eventDates**: **["2026-08-24", "2026-08-25", "2026-08-26"]**.

****Step 2: Engine Initialization & Eligibility Check (Backend)****

1. Backend creates **Booking** record with status **PENDING**.
2. Engine validates requested Primary Vendor using **100% Date Coverage Rule**:
 - Checks if vendor is active & approved by Admin.
 - Checks if vendor has remaining lead balance (**remainingLeads > 0**).
 - Verifies vendor is free on **100% of all selected dates**. If blocked on any single date $\rightarrow$ **eligible = false**.
3. If eligible, creates **BookingVendorAssignment**: **role = PRIMARY, status = PENDING, timeoutAt = now + 2 hours**.

****Step 3: Standby Discovery & Priority Scoring****

1. Engine queries all other active approved vendors in the same category.
2. Filters out vendors blocked/booked on **any requested date**.
3. Calculates **Priority Score** for candidate vendors:

$$\text{Score} = (40\% \times \text{Availability}) + (20\% \times \text{Plan Tier}) + (15\% \times \text{Rating}) + (15\% \times \text{Proximity}) + (10\% \times \text{Response Rate})$$

4. Creates **STANDBY** assignments and broadcasts emails to candidates.

****Step 4: Decision Pathways & SLA Timeout****

- **Acceptance Pathway**: Vendor accepts $\rightarrow$ Booking moves to **WAITING_PAYMENT** $\rightarrow$ Lead balance deducted $\rightarrow$ Customer alerted to pay advance.
- **Rejection/Timeout Pathway**: Vendor rejects with mandatory reason OR 2-hour SLA expires $\rightarrow$ Engine promotes top **AVAILABLE** or **PENDING** standby vendor to **PRIMARY** $\rightarrow$ Dispatches alert.
- **Customer Swap Pathway**: Customer selects alternative vendor $\rightarrow$ Engine demotes old primary to standby $\rightarrow$ Promotes chosen vendor to primary $\rightarrow$ Dispatches role update emails.

****Step 5: Payment Gateway & Final Date Lock****

1. Customer pays advance via Razorpay.
 2. Server verifies HMAC-SHA256 signature (**razorpay_order_id, razorpay_payment_id, razorpay_signature**).
 3. Booking status updates to **ADVANCE_PAID / CONFIRMED**.
 4. System automatically locks all requested dates on vendor calendar and generates PDF invoice.
-

■ 2. System Architecture Diagram

```
graph TD
    subgraph Client_Layer [Frontend - Next.js 14 App Router]
        CustomerUI["Customer Portal (Search, Book, Pay, Track)"]
        VendorUI["Vendor Dashboard (Leads, Availability, Confirmations)"]
        AdminUI["Admin Panel (Audits, Assignments, Lead Ledger)"]
    end

    subgraph API_Gateway [Backend - NestJS Framework]
        AuthModule["JWT Auth & Guards (Role-Based Access Control)"]
        BookingEngine["Smart Booking & Dispatch Engine"]
        AvailabilityEngine["Multi-Date Availability Verification Engine"]
        PaymentService["Razorpay Payment & Verification Engine"]
        NotificationModule["Mail & Activity Audit Log Service"]
    end

    subgraph Data_Storage [Data & Storage Layer]
        PostgreSQL["PostgreSQL Database (Prisma ORM)"]
        Cloudinary["Cloudinary Media CDN"]
        SMTP["SMTP Mail Gateway (Nodemailer)"]
    end

    CustomerUI -->|REST API / Axios| AuthModule
    VendorUI -->|REST API / Axios| AuthModule
    AdminUI -->|REST API / Axios| AuthModule

    AuthModule --> BookingEngine
    AuthModule --> AvailabilityEngine
    AuthModule --> PaymentService

    BookingEngine --> PostgreSQL
    AvailabilityEngine --> PostgreSQL
    PaymentService --> PostgreSQL
    PaymentService -->|Razorpay SDK| RazorpayGateway["Razorpay Test/Live API"]

    NotificationModule --> SMTP
    VendorUI -->|Image Upload| Cloudinary
```

3. Technical Stack Breakdown

Layer	Technology	Purpose
Frontend Framework	Next.js 14 (App Router)	Single Page Application with Server-Side Rendering (SSR) & Client Components
Frontend Styling	Tailwind CSS / Custom CSS	Glassmorphism UI, Responsive Grids, Premium Gold/Maroon Aesthetics
State & Animations	Framer Motion & Zustand	Smooth UI transitions, active state management, toast notifications (Sonner)
Backend Framework	NestJS (Node.js/TypeScript)	Modular, scalable enterprise backend architecture with Dependency Injection
Database ORM	Prisma ORM	Type-safe database queries, schema migrations, and relational modeling
Database Engine	PostgreSQL	Relational Database for ACID-compliant transactions, lead balances, and logs
Payment Gateway	Razorpay API	Advance & Balance payment processing with automated signature verification & test fallback
Media Storage	Cloudinary CDN	Cloud asset management for vendor portfolios, logos, and package images
Email Gateway	Nodemailer (SMTP)	Transactional emails for primary requests, standby broadcasts, and payment alerts

■ 4. Database Schema Overview (Entity Relationship)

```
erDiagram
    USER ||--o{ BOOKING : places
    USER ||--o{ VENDOR : profile
    VENDOR ||--o{ PACKAGE : offers
    VENDOR ||--o{ VENDOR_AVAILABILITY : blocks
    VENDOR ||--o{ BOOKING_VENDOR_ASSIGNMENT : receives
    BOOKING ||--o{ BOOKING_VENDOR_ASSIGNMENT : tracks
    BOOKING ||--o{ PAYOUT : generates

    USER {
        string id PK
        string email
        string name
        enum role "CUSTOMER | VENDOR | ADMIN"
    }

    VENDOR {
        string id PK
        int frontendVendorId
        string businessName
        string badge "BRONZE | SILVER | GOLD | PLATINUM"
        enum status "PENDING | APPROVED | REJECTED"
    }

    PACKAGE {
        string id PK
        string title
        decimal price
        string categoryId FK
    }

    BOOKING {
        string id PK
        string bookingNumber
        datetime eventDate
        string_array eventDates
        decimal totalAmount
        decimal amountPaid
        enum status "PENDING | WAITING_PRIMARY_VENDOR | ADVANCE_PAID | CONFIRMED"
    }

    BOOKING_VENDOR_ASSIGNMENT {
        string id PK
        enum role "PRIMARY | STANDBY"
        enum status "PENDING | ACCEPTED | REJECTED | AVAILABLE | NOT_AVAILABLE"
        int priority
    }
```

■ 5. College Presentation Structure (10-Slide PPT Guide)

■ Slide 1: Title Slide

- **Project Title:** Barati Gharati — Smart AI-Driven Wedding Management Platform

- **Subtitle:** Full-Stack Multi-Vendor Booking & Multi-Date Scheduling Engine
- **Presented By:** [Your Name & Roll Number]
- **Department/Branch:** Computer Science & Engineering / IT

■ Slide 2: Problem Statement & Motivation

- **Fragmented Industry:** Couples struggle to find verified vendors and manage multi-day event schedules (Haldi, Mehendi, Wedding).
- **Communication Delays:** Unresponsive primary vendors block bookings for days without fallback options.
- **Double Booking Risks:** Vendors accepting overlapping dates manually leading to last-minute cancellations.

■ Slide 3: Proposed Solution

- **Unified 3-Tier Ecosystem:** Seamless interaction between Customer, Vendor, and Administrator.
- **Smart Engine:** Automated Primary & Standby vendor assignment with 2-hour SLA response timeout.
- **100% Multi-Date Coverage:** Instant cross-verification across all event dates.
- **Transparent Payments & Ledger:** Razorpay integration with advance booking confirmation.

■ Slide 4: System Architecture

- **Frontend:** Next.js 14, TypeScript, Framer Motion, Tailwind CSS
- **Backend:** NestJS REST API, Prisma ORM, PostgreSQL
- **Integrations:** Razorpay Payment Gateway, Cloudinary CDN, Nodemailer SMTP

■ Slide 5: Core Workflow — Customer Journey

1. Browse Vendors & Filter by Category / Price.
2. Select Multiple Event Dates (Haldi, Wedding, etc.).
3. Choose Package & Submit Booking Request.
4. View Cross-Vendor Recommendations & Payment Details.
5. Track Vendor Acceptance & Pay Booking Advance.

■ Slide 6: Core Workflow — Vendor & Smart Engine

- **Primary Assignment:** Direct notification to requested vendor.
- **Standby Broadcast:** Instant notification to matching standby vendors.
- **Mandatory Rejection Reason:** Modal with quick presets (**Already booked, Budget mismatch**).
- **Vendor Swap:** Graceful demotion & promotion logic without data loss.

■ Slide 7: Multi-Date Availability Engine Algorithm

- Explanation of the **All-Dates Availability Algorithm**:

$$\text{Eligibility} = \bigwedge_{i=1}^N \Big(\text{Unblocked}(\text{Date}_i) \wedge \text{Unbooked}(\text{Date}_i) \Big)$$

- If any Date_i is false $\rightarrow$ Vendor is flagged **NOT_AVAILABLE**.

■ Slide 8: Database & Data Modeling

- Relational mapping between **User**, **Vendor**, **Package**, **Booking**, **VendorAssignment**, and **Payout**.

■ Slide 9: Security, Admin Audit & Error Handling

- **Role-Based Access Control (RBAC)**: JWT authentication with guards for Admin/Vendor/Customer.
- **Admin Dashboard**: Full visibility into rejection reasons, manual vendor assignments, and payout releases.
- **Resilient Fallback**: Automatic mock payment gateway mode for test environments.

■ Slide 10: Conclusion & Future Scope

- **Key Deliverables Accomplished**: Working full-stack application, automated vendor engine, multi-date scheduling, Razorpay payments.
- **Future Work**: AI chatbot for wedding budgeting, real-time WebSockets chat between customer & vendor, mobile app (React Native / Flutter).

Generated for Barati Gharati Project Architecture & Academic Documentation.